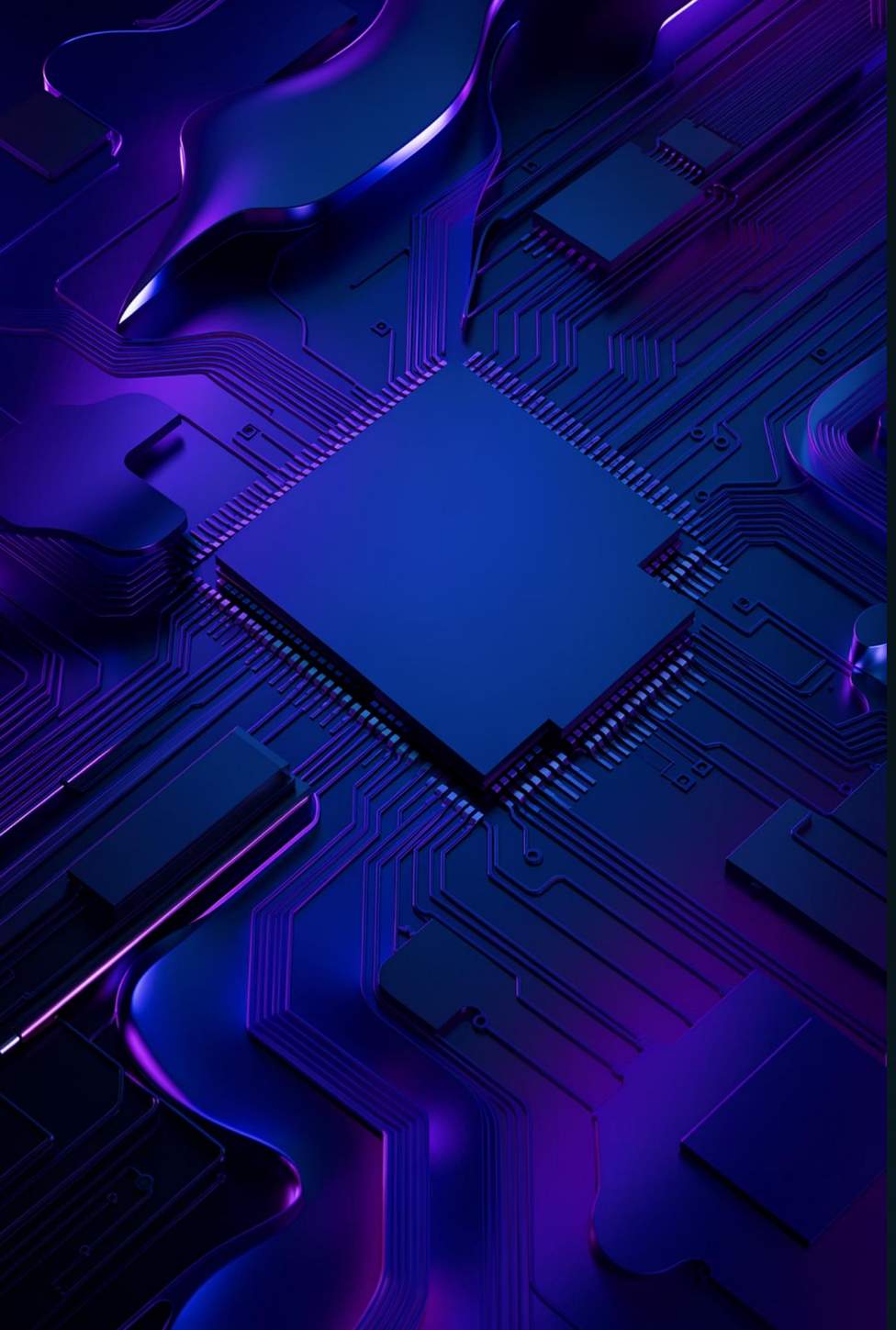


University of Sheffield

Hardware Acceleration for Cluster Fault Tolerance in Hybrid CMOS/Non-CMOS Memories

Yuqi Guo — 230184273

Supervisor: Mr. Neil Powell | Second Marker: Dr. Mohammad Eissa

Project Overview

01

Motivation and Problem

Hybrid memories face spatially correlated cluster faults

02

Project Goal and Objectives

FPGA-based hardware acceleration platform design

03

Methodology and Experimental Setup

Two-phase evaluation framework

04

Main Innovation

Probabilistic Fault Injection Engine

05

Results

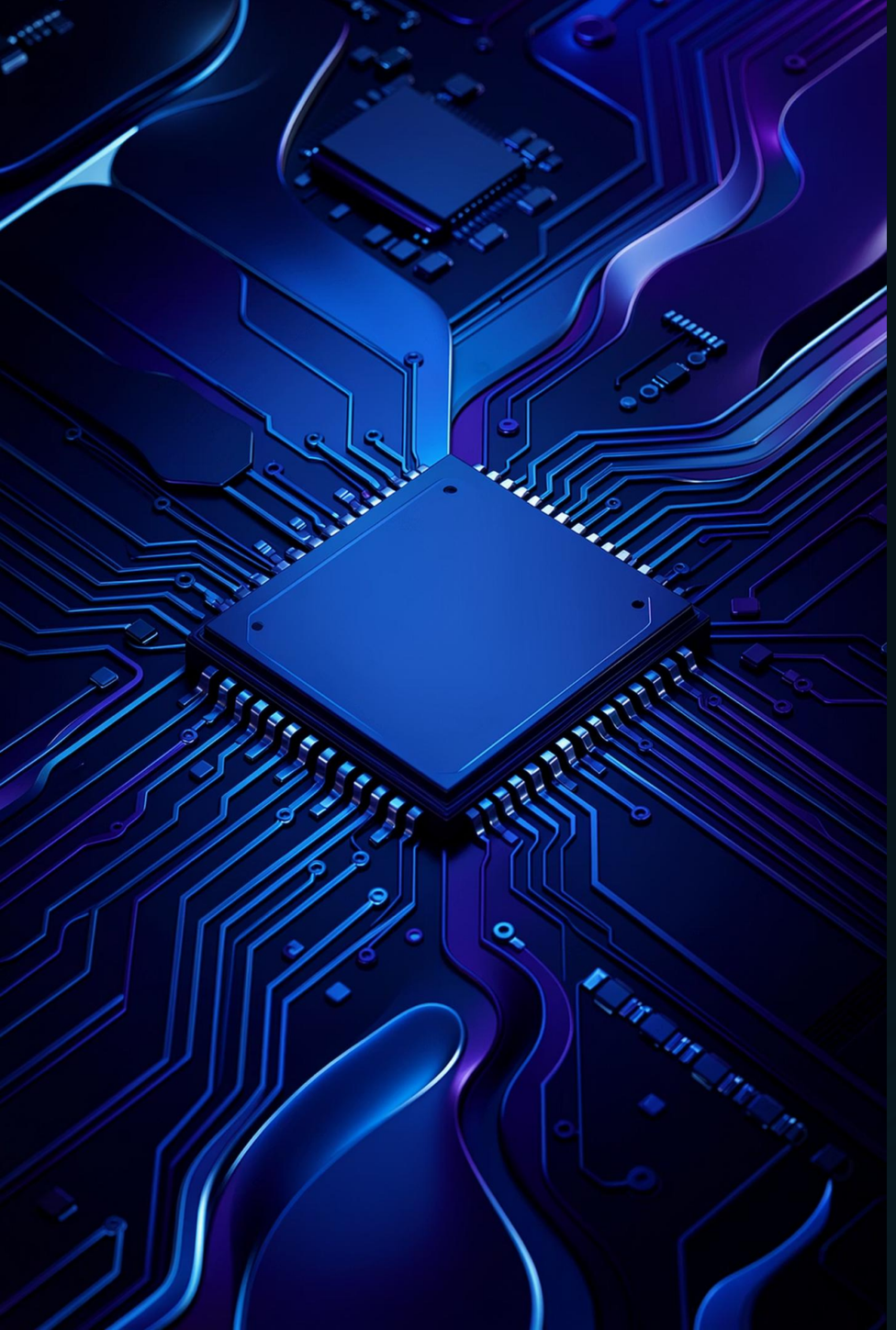
Random single-bit and cluster burst analysis

06

Contributions and Future Work

Extensible architecture and findings

Core Contribution: First hardware-validated multi-algorithm RRNS benchmark on FPGA, with novel probabilistic fault injection engine



University of Sheffield

Background and Research Context

Why This Project Matters

Hybrid CMOS/non-CMOS memories offer high density and low cost, but they are vulnerable to **spatially correlated cluster faults**. Traditional ECC schemes are often evaluated for independent random errors, not realistic burst faults. Previous studies mainly relied on simulation; **hardware-validated comparison** was still missing.

Core Research Question

Which ECC design gives the best trade-off between fault tolerance, latency, area, power, and storage efficiency on real FPGA hardware?

Project Goal and Objectives

Project Goal

To design and implement an **FPGA-based hardware acceleration platform** for fair benchmarking of multiple ECC algorithms under realistic cluster-fault conditions.

Implementation Configurations

- 2NRM-RRNS Parallel
- 2NRM-RRNS Serial
- 3NRM-RRNS
- C-RRNS-MLD
- C-RRNS-MRC
- RS(12,4)

Fault Tolerance

Burst error correction capability

Decoding Latency

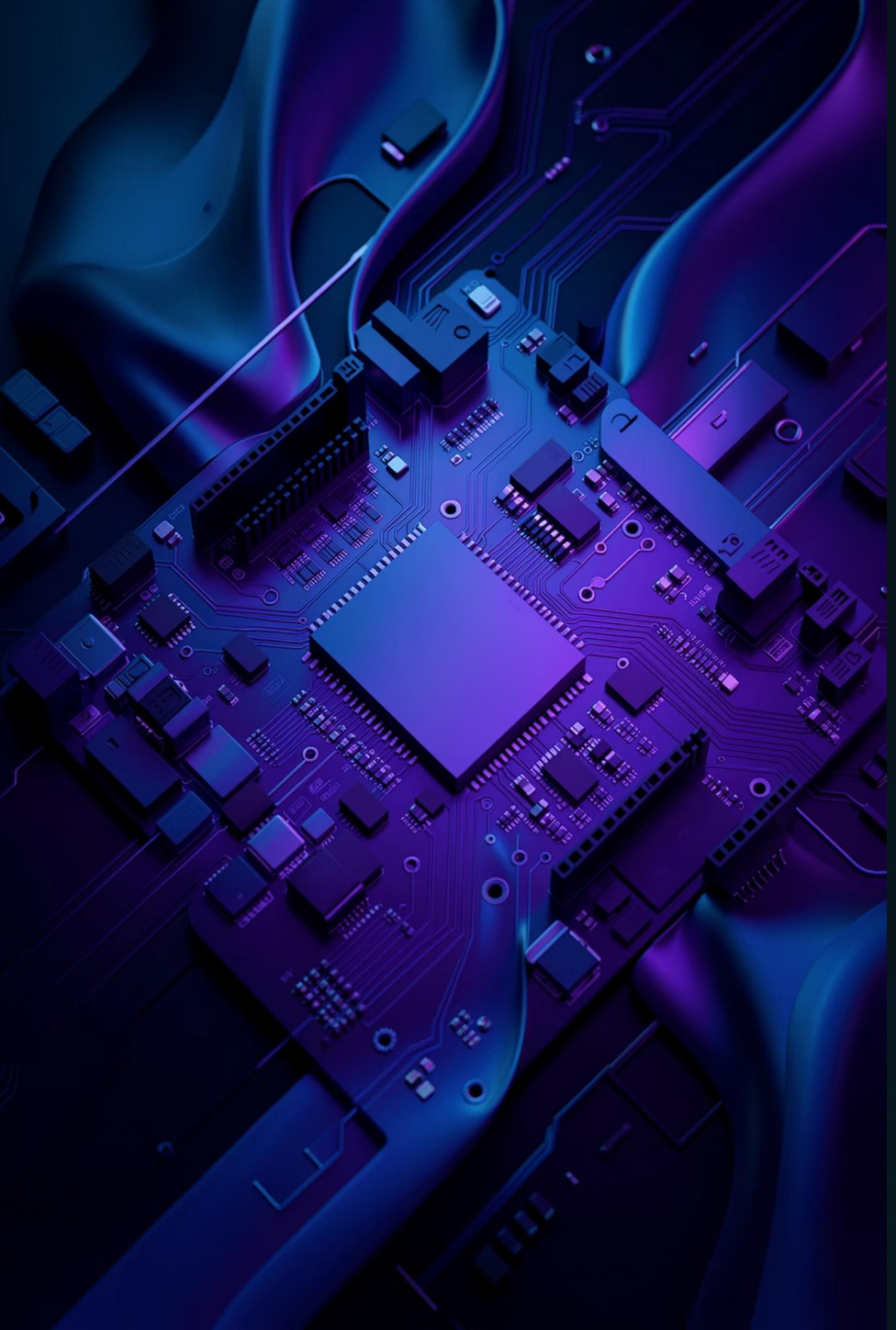
Processing time per frame

FPGA Resources

Area and power consumption

Storage Efficiency

Code rate and overhead



Methodology and Experimental Setup

Phase 1: MATLAB Simulation

Theoretical baseline analysis

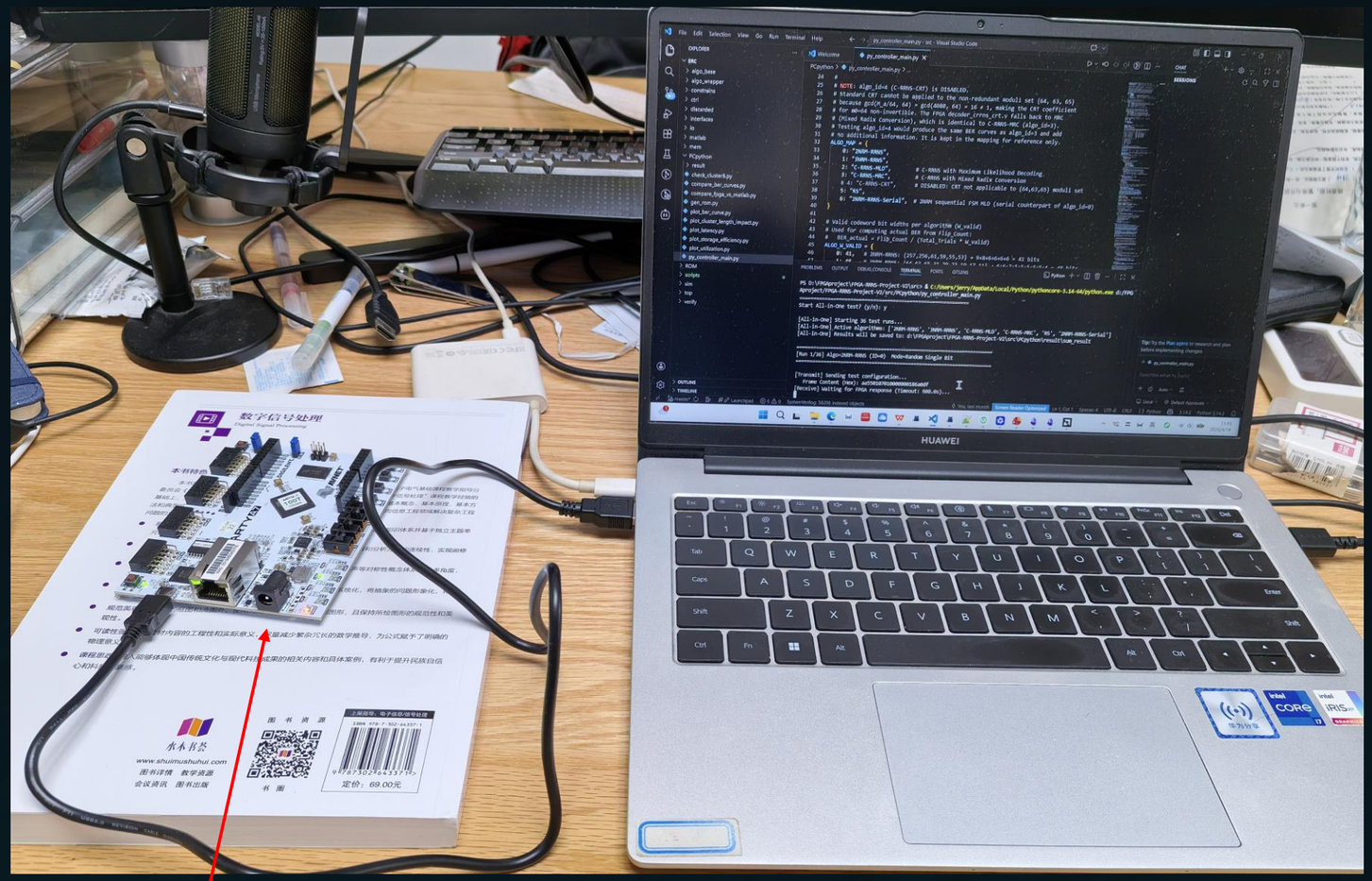
Phase 2: FPGA Implementation

Hardware validation

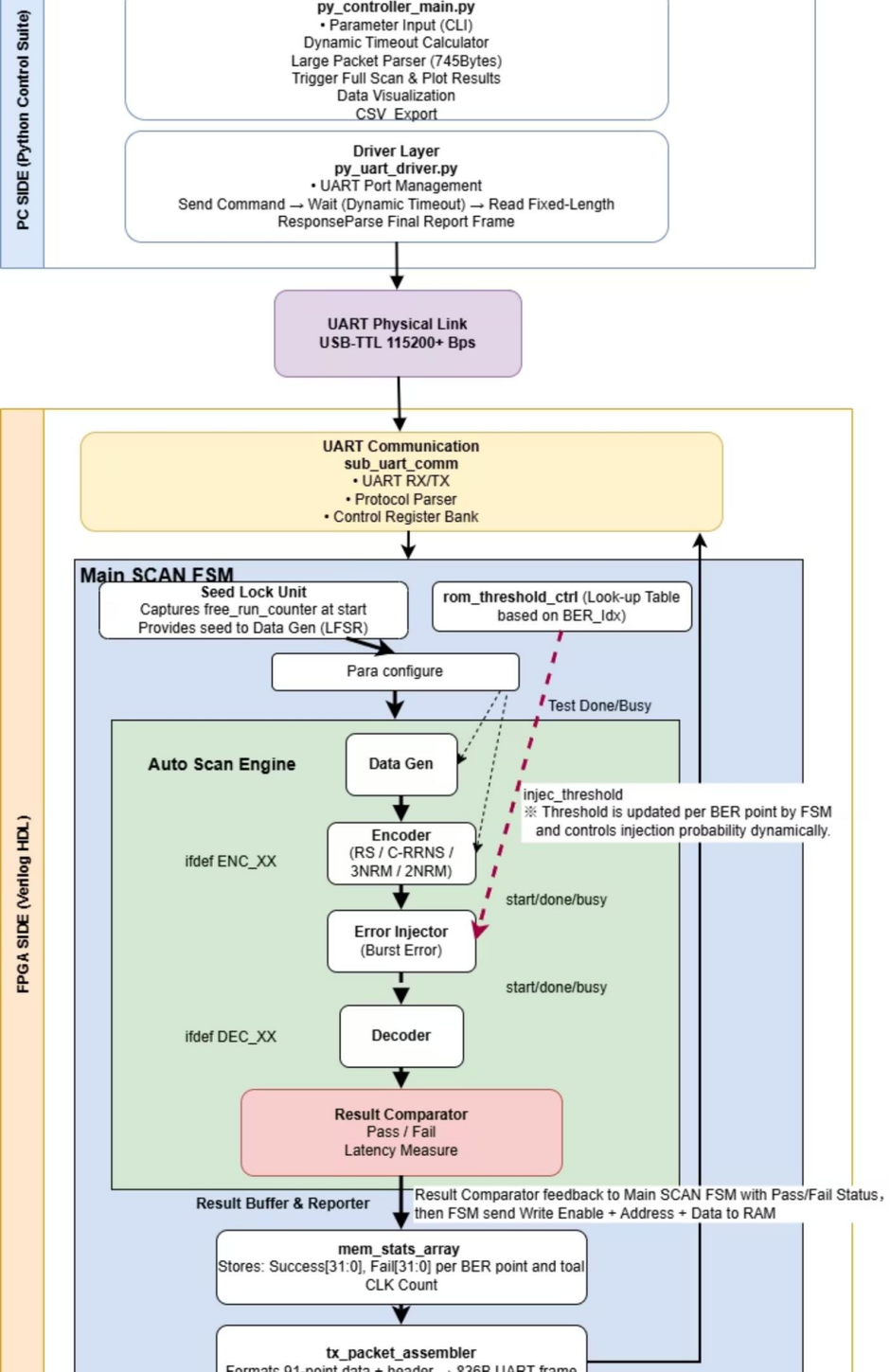
Test Settings

- Device: Xilinx Artix-7 xc7a100t (Arty A7-100T)
- System Clock: 50 MHz
- Fault Injection Modes: Random single-bit and cluster burst (length 1-15)

FPGA test platform under working



Xilinx Artix-7 xc7a100t (Arty A7-100T)



FPGA Platform Architecture

Key Idea

The PC controls configuration and visualization, while the FPGA autonomously performs the full BER sweep and uploads consolidated results through UART. This architecture enables **scalable, hardware-validated testing** with minimal host intervention.

Single-Algorithm-Build: Each synthesis run instantiates exactly one codec → ensures fair, interference-free resource comparison

Main Innovation: Probabilistic Fault Injection Engine

1

32-bit Galois LFSR

Pseudo-random source for fault generation

Period: $2^{32}-1 \approx 4.3\times10^{9}$ cycles

2

Dual Injection Modes

Random single-bit and cluster burst support

Random single-bit + Cluster burst (L=1~15)

3

Threshold Control

BER controlled by dynamic comparison

101 BER points, 0%~10%, step 0.1%

4

Precomputed Masks

Boundary-safe burst patterns in ROM

Only 2 BRAMs, support 1~1000,000 samples/point configurable

Why It Matters

This makes the platform **resource-efficient, reproducible, and reusable** for benchmarking different ECC algorithms.

Why the Test Platform Is Strong

Fair Comparison

All algorithms share the same test infrastructure and clock conditions

Hardware Validated

Results measured on Xilinx Artix-7 FPGA, not simulation alone

Low Overhead

Probabilistic injector uses only two BRAMs

Scalable Experiments

101 BER points from 0% to 10%, 100,000 samples per point
Support 1~1000,000 samples per point configurable

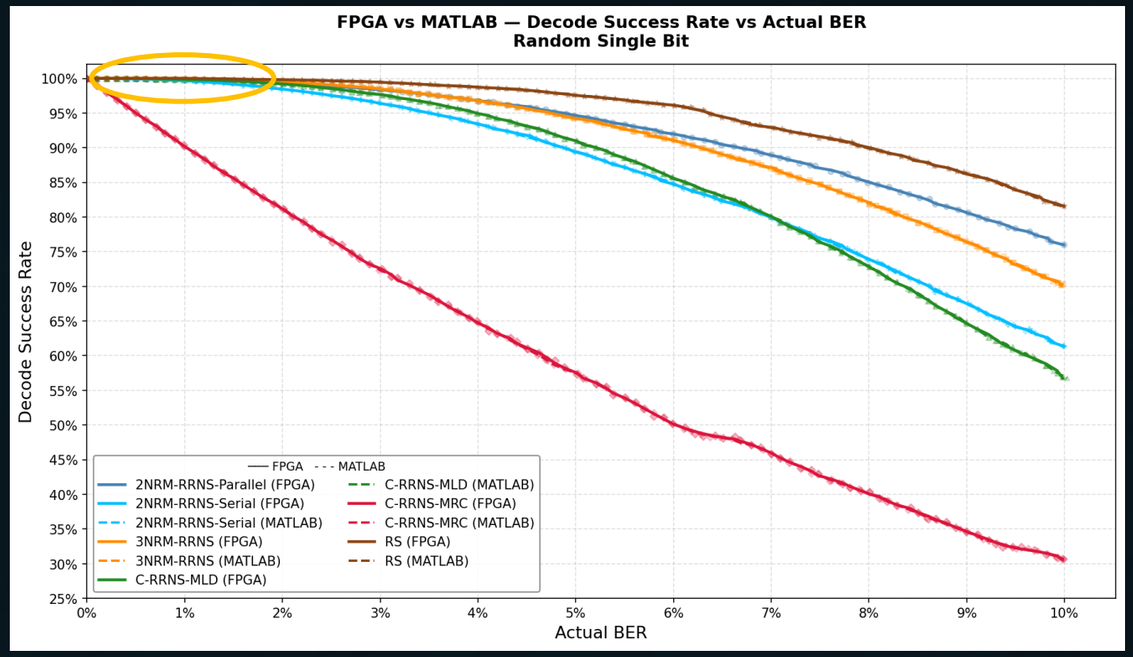
Extensible Architecture

New ECCs added with wrapper interface

Cross-Validated

FPGA curves closely match MATLAB results

Random Single-Bit Results



100000 samples each BER performed in this project

Main Observations

Model Agreement

FPGA and MATLAB curves show close agreement across all BER points

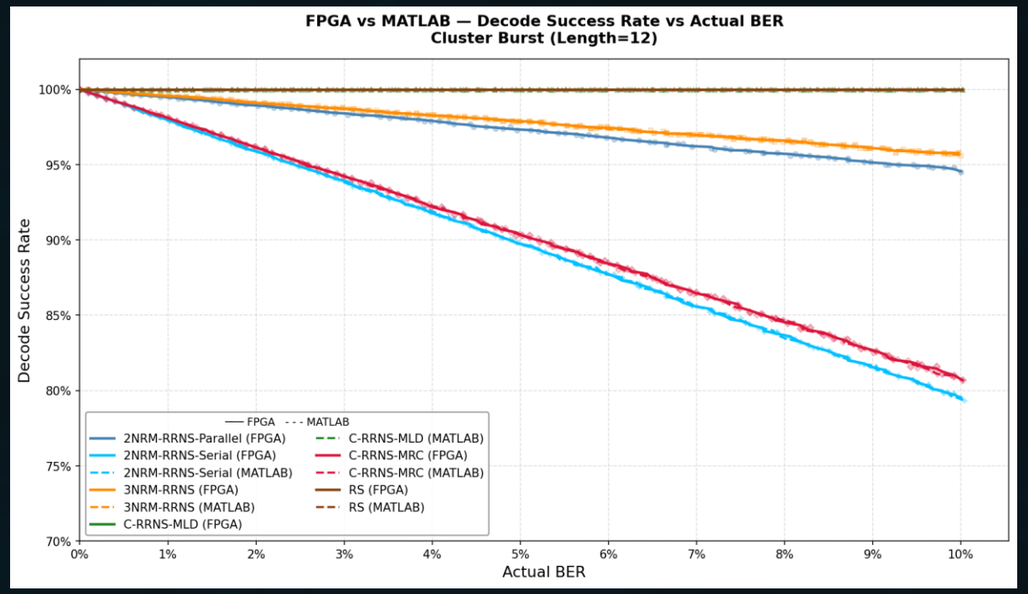
Differential Performance

Correcting ECCs remain strong at low BER, while C-RRNS-MRC drops fastest at higher error rates

Parallel vs Serial

Parallel decoder (73 cycles/trial) shows higher apparent success due to LFSR correlation

Cluster Burst Results at Representative Length L=12



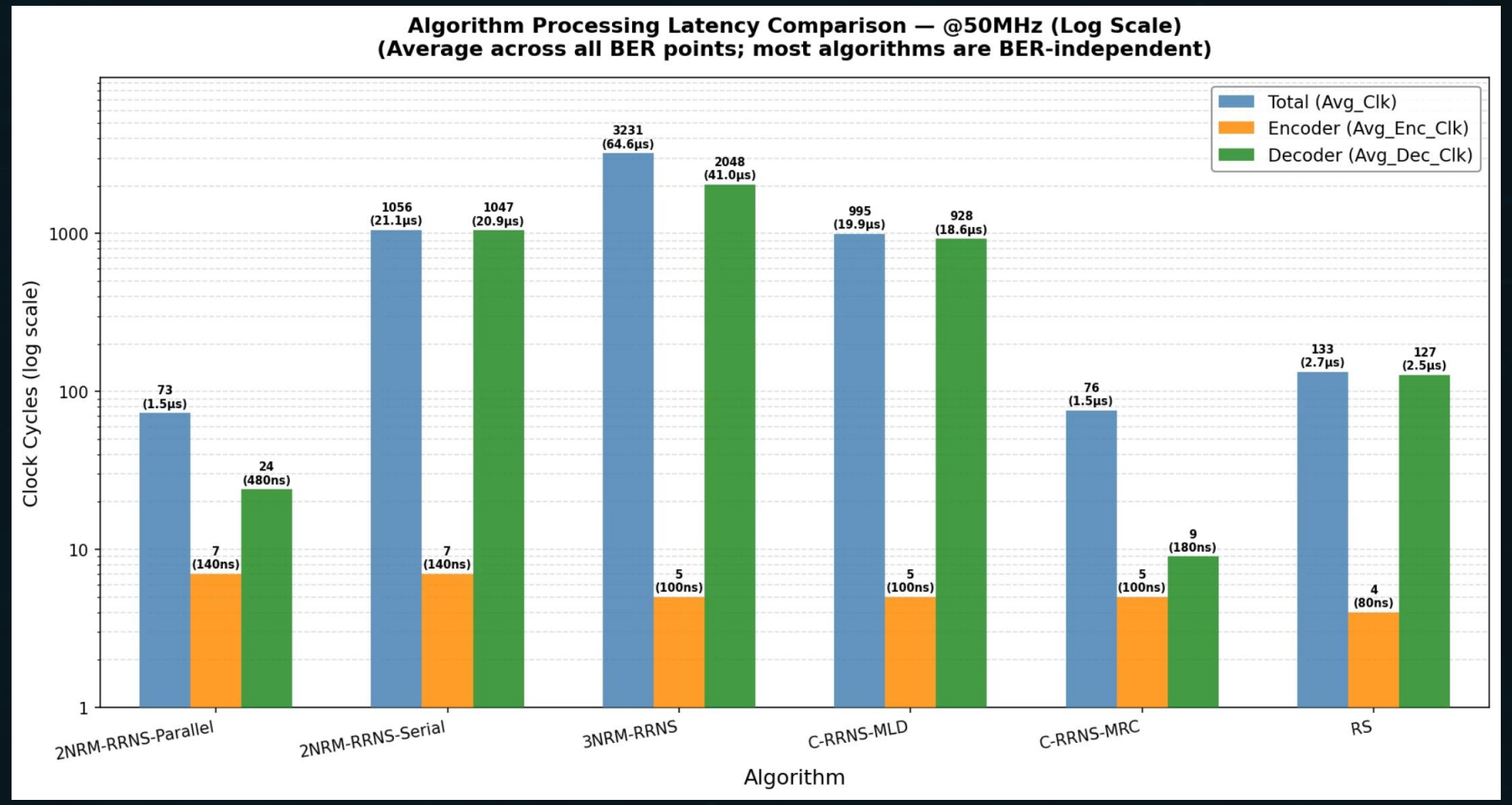
Key Findings

Top Performers

C-RRNS-MLD & RS(12,4): **100% decode success** at BER=10%, L=12

Model Dependence

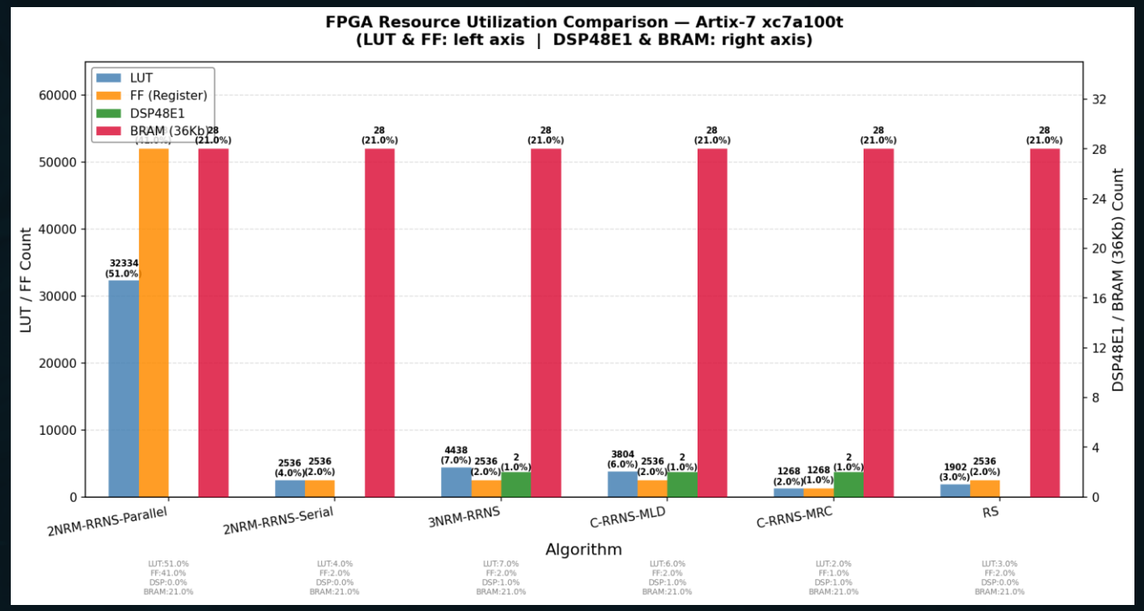
Burst-fault ranking differs from random-bit testing, showing why a realistic fault model is necessary



Latency Comparison

Performance Analysis

- **2NRM-RRNS Parallel: 43× faster than Serial** (24 vs 1047 decoder cycles) At 13× higher LUT utilization (51% vs 4%)
- Parallel implementation achieves major acceleration over serial version
- Results expose the **resource-latency trade-off** of parallel MLD architectures



Resource and Storage Comparison

Key Trade-offs

2NRM-Parallel

Achieves high speed at highest LUT/FF and power cost

2NRM-RRNS

Most storage-efficient option for overhead-sensitive designs

C-RRNS-MLD

Provides strongest protection with balanced resource usage

Quantitative Comparison of Key Results

Algorithm	Max Burst L	Dec. Cycles	LUT	Power (W)	Storage
C-RRNS-MLD	14	928	6%	0.232	26.2%
C-RRNS-MRC	NA	9	2%	0.216	26.2%
RS(12,4)	13	127	3%	0.216	33.3%
3NRM-RRNS	11	2048	7%	0.242	33.3
2NRM (Parallel)	8	24	51%	0.58	39%
2NRM (Serial)	7	1047	4%	0.223	39%

Most Reliable: C-RRNS-MLD provides the strongest burst-fault tolerance

Fastest: 2NRM-RRNS Parallel decoder achieves the lowest decode latency

Most Compact: 2NRM-RRNS offers the best storage efficiency

Application scenario recommendations

Scenario	Primary Constraint	Recommended	Rationale
High-reliability systems (aerospace, medical)	Fault tolerance	C-RRNS MLD or RS	100% recovery, L=14/13
Storage-constrained devices (edge computing, IoT)	Codeword overhead	2NRM-RRNS (Parallel)	39.0% efficiency, 10.96 Mbps
Latency-sensitive systems (real-time memory controllers)	Processing speed	RS(12,4) or 2NRM-RRNS (Parallel)	RS: 6.02 Mbps with t=4 symbol correction; 2NRM-Parallel: 10.96 Mbps with t=2 correction
Resource constrained FPGAs (low-cost devices)	LUT utilization	2NRM-RRNS (Serial)	
Balanced general-purpose use	All dimensions	RS(12,4)	Moderate latency (127 cycles), highest correction capability (t=4), 33.3% storage efficiency

Contributions and Future Work

Contributions

FPGA Evaluation Platform

First reusable FPGA evaluation platform for multi-algorithm RRNS benchmarking

Fault Injection Engine

Novel probabilistic fault injection engine: 2 BRAMs, 100K samples/point, arbitrary sample count

Trade-off Quantification

First hardware quantification of 2NRM-RRNS parallel/serial trade-off: 43× latency, 13× LUT

Future Work

Expand Evaluation Scope

Evaluate wider data words and additional ECC variants

Enhance Randomization

True Random Number Generator (TRNG)

ASIC Migration

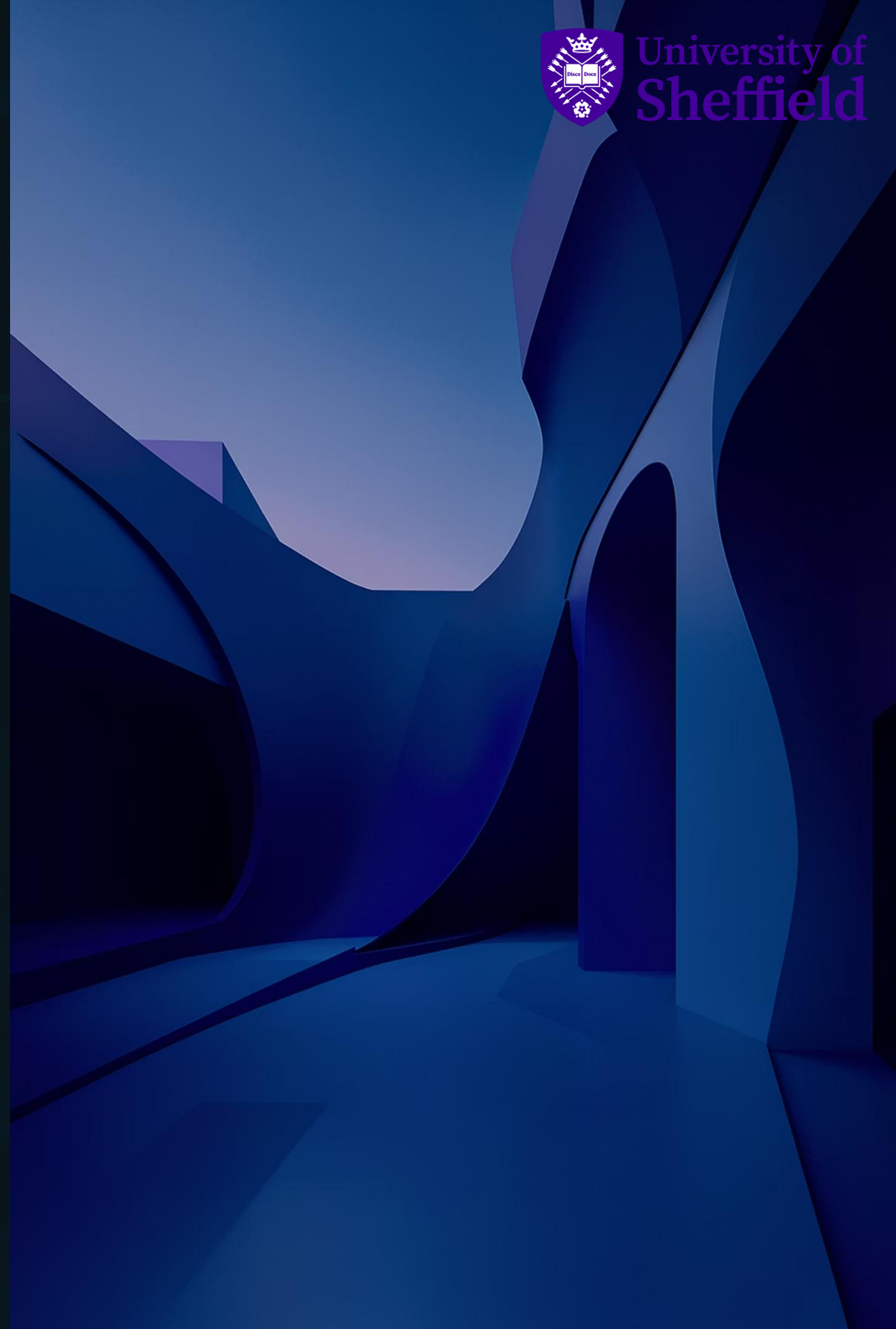
Migrate framework toward ASIC or real hybrid memory devices

Thanks

Yuqi Guo

Sheffield University

May , 2026



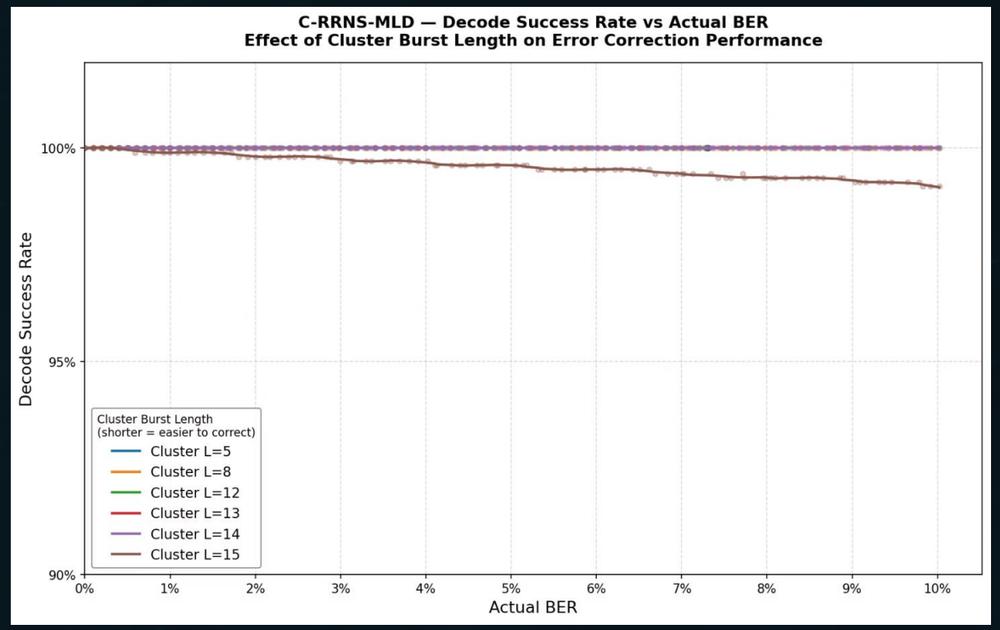
Appendix A: Fault Injection Algorithm

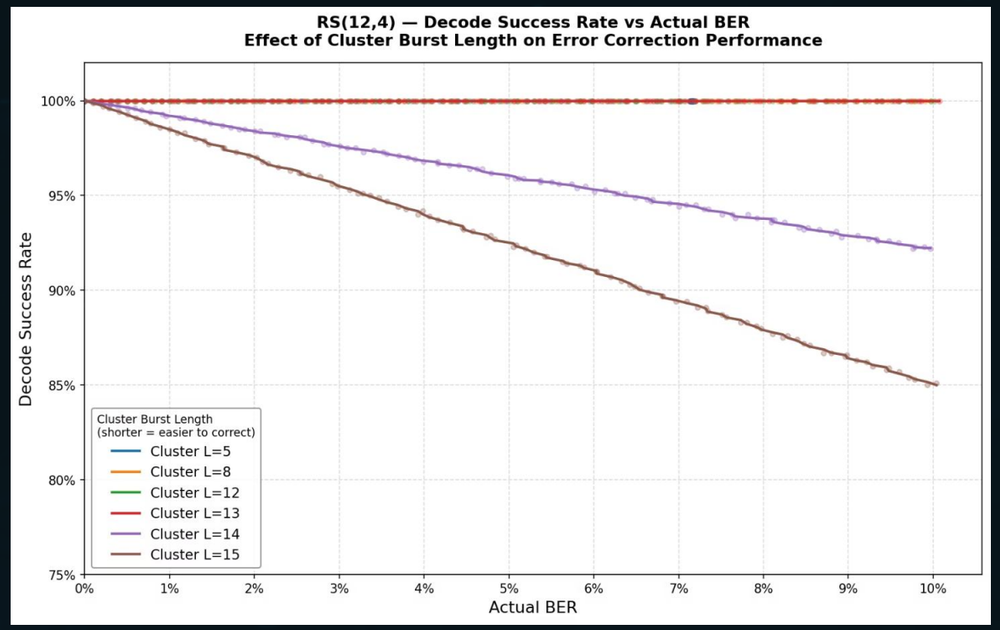
Probabilistic Injection Procedure

1. **Generate Encoded Codeword**
   Create encoded data word for testing
2. **Lock Random Seed**
   Lock one random seed for full BER sweep
3. **Compute Threshold**
   For each BER point, compute injection threshold
4. **Generate Errors**
   Use 32-bit Galois LFSR to decide error injection
5. **Apply Mode**
   Test valid bits independently or apply ROM-based burst mask
6. **Decode & Verify**
   Decode and compare with original data to record results

Controlled Evaluation: This appendix demonstrates the fault injector is not a simple random flip unit, but a **controlled and reproducible hardware evaluation algorithm** with deterministic behavior.

Appendix B: Burst-Length Impact





Backup Results

These curves support the final claim that **C-RRNS-MLD** provides the best extreme cluster-fault tolerance, with **RS** as the strongest balanced alternative.